



UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Cuajimalpa

Sistemas Distribuidos

Práctica 2: Calculadora Remota Mejorada

Arquitectura Cliente-Servidor con Frontend y Cliente Intermedio

Integrantes:

Aaron Rodrigo Ramos Reyes
Ismael Valencia Reyes
Fabian Emiliano Castañeda Juvera

Ciudad de México, 26 de mayo de 2026

Índice

1. Introducción	2
2. Mejoras Realizadas	2
3. Diagrama de Componentes	2
3.1. Explicación de las Relaciones	2
4. Explicación del Código	3
4.1. Servidor (C++)	3
4.2. Cliente Consola (C++)	4
4.3. Interfaz Gráfica (Python)	4
5. Flujo de Uso	4
5.1. 1. Obtener la IP del Servidor	4
5.2. 2. Levantar el Servidor	4
5.3. 3. Compilar el Cliente C++	4
5.4. 4. Ejecutar el Frontend Python	4
6. Evidencias	5
7. Conclusiones	5

1. Introducción

En esta segunda práctica se mejoró significativamente una aplicación cliente-servidor para realizar operaciones aritméticas básicas (+, -, *, /) de forma distribuida.

El objetivo principal fue reforzar los conceptos de **arquitectura distribuida**, **separación de responsabilidades** y **comunicación entre procesos**. Se mantiene un servidor robusto en C++, pero se modificó la arquitectura para que el frontend en Python funcione únicamente como interfaz gráfica, delegando toda la lógica de red al cliente en C++ mediante llamadas a `subprocess`.

Esta nueva estructura permite una mejor modularidad, facilitando el mantenimiento y la escalabilidad del sistema.

2. Mejoras Realizadas

Respecto a la práctica anterior, se implementaron las siguientes mejoras:

- **Separación clara de responsabilidades:** El frontend Python ya no se conecta directamente al servidor. Ahora actúa solo como interfaz gráfica.
- **Cliente C++ como capa intermedia:** Se modificó para aceptar parámetros por línea de comandos, permitiendo su ejecución desde Python.
- **Soporte nativo de múltiples clientes:** El servidor maneja concurrentemente múltiples conexiones mediante hilos (`std::thread`).
- **Mejor experiencia de usuario:** Se agregó un campo editable para la IP del servidor directamente en la interfaz gráfica.
- **Robustez del servidor:** Mejoras en el manejo de buffers, parsing de datos y gestión de errores.
- **Interoperabilidad entre lenguajes:** Uso de `subprocess` para comunicar Python con el ejecutable C++.
- **Facilidad de conexión:** Cualquier máquina en la misma red puede conectarse usando la IP del servidor.

3. Diagrama de Componentes

3.1. Explicación de las Relaciones

El sistema está compuesto por tres componentes principales:

- **Frontend Gráfico (Python):** Solo interfaz de usuario. No realiza conexiones de red.
- **Cliente C++:** Capa intermedia responsable de toda la comunicación TCP con el servidor.
- **Servidor Multihilo (C++):** Componente central que procesa las solicitudes.

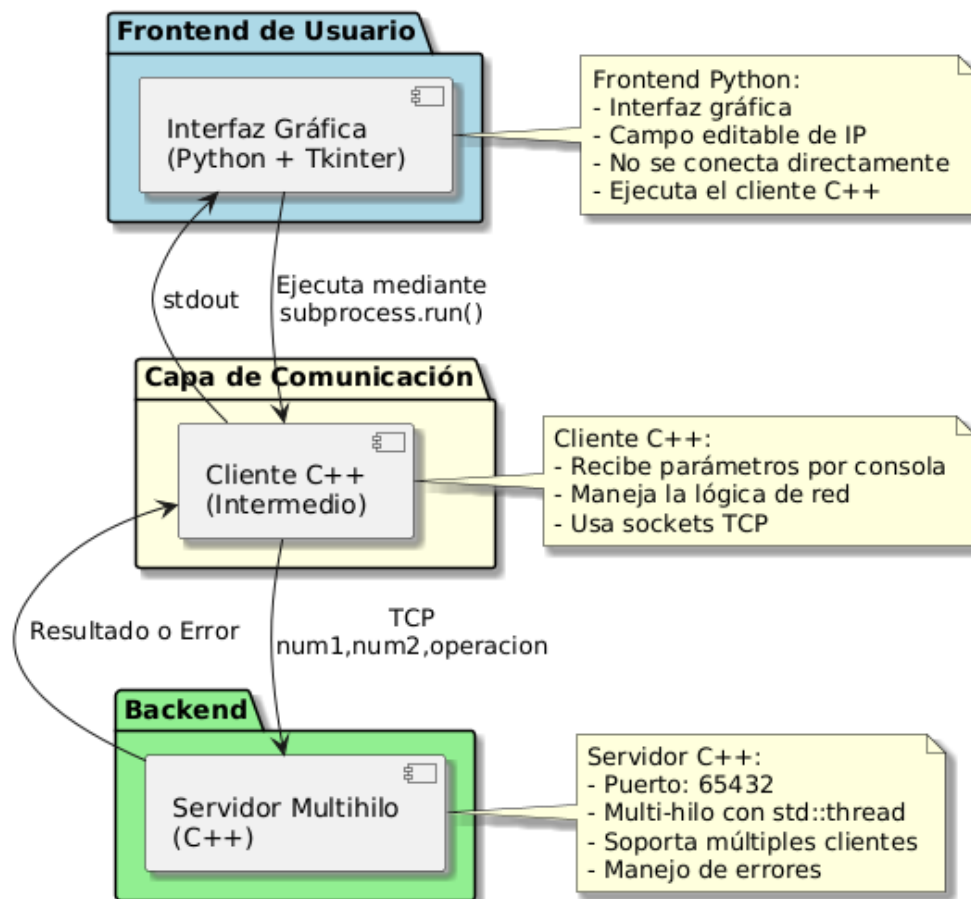


Figura 1: Diagrama de Componentes de la Calculadora Remota - Práctica 2

El flujo es: ****Python → Cliente C++ (subprocess) → Servidor (TCP)****. La respuesta sigue el camino inverso.

4. Explicación del Código

4.1. Servidor (C++)

El servidor es el núcleo del sistema. Utiliza sockets TCP y un modelo multi-hilo para atender múltiples clientes simultáneamente.

Las partes clave incluyen:

- Configuración del socket con `SO_REUSEADDR` | `SO_REUSEPORT`.
- `bind()` en todas las interfaces (`INADDR_ANY`) en el puerto 65432.
- Bucle principal con `accept()` y creación de hilos para cada cliente.
- Función `atender_cliente()`: Lee los datos, parsea el formato `num1,num2,op`, realiza el cálculo y devuelve la respuesta.
- Manejo seguro de errores y cierre correcto de sockets.

4.2. Cliente Consola (C++)

Este componente fue rediseñado para ser invocado desde otros programas. Sus características principales son:

- Recepción de parámetros por línea de comandos: IP, num1, num2 y operación.
- Creación de socket TCP y conexión al servidor.
- Envío del mensaje en formato texto.
- Recepción y salida limpia de la respuesta (ideal para capturar con Python).

4.3. Interfaz Gráfica (Python)

Desarrollada con Tkinter, su rol es exclusivamente visual. Destacan:

- Campo editable para la IP del servidor.
- Teclado numérico y botones de operación.
- Ejecución del cliente C++ mediante `subprocess.run()` en un hilo separado.
- Captura de la salida estándar y manejo de errores amigable.

5. Flujo de Uso

5.1. 1. Obtener la IP del Servidor

Ejecutar en la máquina del servidor:

```
1 ip a | grep inet
```

O

```
1 hostname -I
```

5.2. 2. Levantar el Servidor

```
1 g++ -std=c++11 -pthread servidor_calculadora.cpp -o servidor
2 ./servidor
```

5.3. 3. Compilar el Cliente C++

```
1 g++ cliente_calculadora.cpp -o cliente
2 chmod +x cliente
```

5.4. 4. Ejecutar el Frontend Python

```
1 python3 frontend_calculadora.py
```

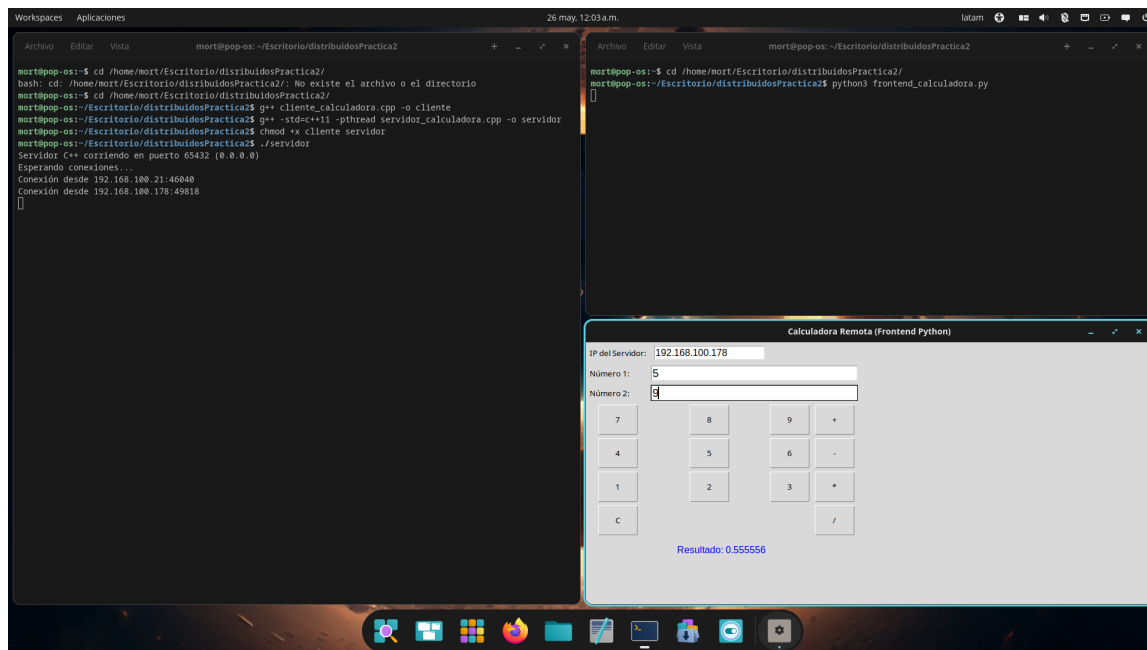


Figura 2: Evidencia de funcionamiento con múltiples clientes

6. Evidencias

En esta evidencia se observa:

- El servidor ejecutándose y recibiendo conexiones desde dos direcciones IP diferentes.
- Un cliente C++ ejecutándose en la misma máquina del servidor.
- Otro cliente (a través del frontend Python) conectándose desde una máquina diferente en la misma red.
- El correcto procesamiento de operaciones y respuestas.

7. Conclusiones

Esta práctica permitió consolidar y profundizar los conocimientos en Sistemas Distribuidos. Entre los principales aprendizajes se encuentran:

- La importancia de una **buena separación de responsabilidades** en arquitecturas distribuidas.
- El uso efectivo de **conurrencia** (hilos) para permitir múltiples clientes simultáneos.
- La interoperabilidad entre lenguajes mediante llamadas a procesos (**subprocess**).
- El diseño de protocolos simples pero efectivos de comunicación cliente-servidor.
- La relevancia del manejo adecuado de errores y la robustez en entornos de red.
- Cómo una interfaz gráfica puede actuar como un "frontend puro" delegando la lógica de red.

El sistema desarrollado demuestra de manera práctica cómo construir aplicaciones distribuidas modulares, escalables y mantenibles, cumpliendo con los objetivos de la asignatura.